Early Attack Identification in the Wild

Minji Kim, Dongeun Lee, Kookjin Lee, Doowon Kim, *Member, IEEE*, Jinpyo Kim, Sangman Lee, and Jinoh Kim, *Senior Member, IEEE*

***Abstract*—While characterizing network connections in their early stage is vital for providing timely responses against network threats, existing methods become less attractive due to the requirement of complete connection information (thus unable to make timely identification) or packet payload inspection (therefore limited to unencrypted packets under no privacy regulation). To this end, this paper takes an approach of *packet stream analysis* referencing statistical information of packet sequences, requiring *neither* packet inspection *nor* complete connection information. To enable practical packet stream analysis, there exist several challenges, such as *out-of-order packet sequences* introduced by network dynamics and *class imbalance* with a tiny fraction of attack connections. To overcome these challenges, we design two deep sequence models: 1) a *bidirectional recurrent structure* designed for greater resilience to out-of-order packet streams; and 2) a *pre-training-enabled sequence-to-sequence structure* designed for creating consistent representations from unbalanced class distributions using self-supervised learning. We evaluate the presented deep sequence models using real and synthetic network data collections for extensive experimentation. The experimental results support the feasibility of the proposed models outperforming baseline deep learning models, yielding up to 94.8% (F1 score) only with the first five packets (k=5) from the Internet traffic collection containing a substantial fraction of network flows experiencing out-of-order delivery.**

***Index Terms*—Packet stream, early characterization, out-of-order packets, class imbalance, deep sequence, neural networks.**

I. Introduction

NETWORK-BASED attacks happen increasingly with growing volume and impacts. This becomes more critical with ever-growing connectivity with less secure Internet of Things (IoT) devices through wireless mobile communications. One of the essential functions to protect computing resources in the networking environment is the effective detection of network attacks. The approach to network attack detection can be either host-based or connection-based (i.e., host-independent), and the latter would be an attractive option with relatively low deployment and operation costs [1]. In addition to detection, the capability of timely responses is another vital requirement, so as to minimize detection deficit (the gap between the time to compromise and the time to defend) [2]. That is, the network should be able to prepare and enforce relevant countermeasures against individual attack activities before they are successfully executed. To this end, there would be at least two prerequisites: First, the class of attacks should be identified in the detection time to prepare relevant countermeasures (e.g., based on predefined security policies). Second, the detection process should be done as early as possible while the attack is active and in progress.

Recently a body of studies has employed machine learning (ML) techniques for detecting network attacks [3], [4]. Despite their promising detection performance reported, most (if not all) of previous studies assume the availability of complete connection records that are only available at the connection termination time. For instance, common public datasets (e.g., KDDCup 1999/NSL-KDD [5], UNSW-NB15 [6], and CIC-IDS2017 [7]) provide the complete connection information such as duration, aggregated bytes, and number of packets, through CSV-formatted files. Such reliance confines the capability of attack detection to postmortem analysis rather than live online detection for timely actions. Moreover, a large body of ML-based detection studies targeted anomaly detection (i.e., binary decision whether it is likely an attack or not); attack detection is a more complicated problem and requires the ability to identify the type of attacks (e.g., Web, DoS, and scanning attacks). Deep packet inspection is an effective method for identifying attack classes at the early stage of connections by applying pattern matching with predefined byte sequences ("signatures") [8]. However, any payload inspection approach is extremely expensive for analyzing packet payloads. It is also limited to unencrypted packets only. Moreover, there is a trend of banning payload scanning with an increasing demand on privacy, which renders it unlawful to read even cleartext packets. In this study, we take an approach of *packet stream analysis* referencing statistical information of packet sequences, thus requiring *neither* packet inspection *nor* complete connection information.

For practical packet stream analysis, there exist several intricate challenges. First, the communication network is wild and network dynamics (e.g., routing oscillation) may lead to *out-of-order* packet sequences in collection. For example, packet loss rates are non-negligible in reality, ranging from 0.01% to 20%, even in wired networking settings [9]. Packet reordering refers to the reception of packets in a different order than what were originally sent, and it is inevitable to see packet reordering in packet-switched networks due to several reasons, such as multi-path forwarding routes, parallelism, and

Received 21 May 2023; revised 10 November 2024, 3 May 2025, and 29 August 2025; accepted 10 March 2026; approved by IEEE TRANSACTIONS ON NETWORKING Editor D. Pei. Date of publication 24 March 2026; date of current version 27 March 2026. *(Corresponding author: Jinoh Kim.)*

Minji Kim, Dongeun Lee, Jinpyo Kim, and Jinoh Kim are with the Computer Science Department, East Texas A&M University, Commerce, TX 75428 USA (e-mail: jinoh.kim@etamu.edu).

Kookjin Lee is with the School of Computing and Augmented Intelligence, Arizona State University, Tempe, AZ 85281 USA.

Doowon Kim is with the Department of Electrical Engineering and Computer Science, University of Tennessee, Knoxville, TN 37996 USA.

Sangman Lee is with the Department of AI Cyber Security, Korea University, Sejong-si 30019, South Korea.

Digital Object Identifier 10.1109/TON.2026.3677135

TABLE I
PRIOR RESEARCH ON PACKET STREAM ANALYSIS

Study	Challenges/Goals	Approaches	Dataset (primary)	Early	Input Data
Chen et al. [19] (2017)	No reliance on distribution of flow attributes	CNN with RKHS embeddings	Local collection	✓	Packet statistics (biflow)
Rezaei et al. [20] (2020)	Scarcity of labeled training samples	Multi-task learning with easily obtainable samples (bandwidth and duration)	QUIC (local collection) [21], ISCX2016 [22]	✓	Packet length, time, direction
Meng et al. [23] (2022)	Packet representation	Attention-based encoding and probability-based classification	ISCX2016 [22], DoHBrw2020 [24], USTC2016 [25]	×	Packet header and content
Ahmad et al. [26] (2022)	Early (real-time) detection	One-dimensional CNN with MLP	CIC-IDS2018 [7]	✓	Raw bytes
Zhang et al. [27] (2022)	Autonomous model updates for new classes	Bayesian neural networks and MLP	ISCX2016 [22]	✓	Raw bytes
Yuan et al. [15] (2023)	Noise labels (hard samples)	Cluster-based classification with boundary augmentation	Local collection	×	Packet header and size
Bierbrauer et al. [28] (2023)	Scarcity of training samples and resource restrictions in edge network	Transfer learning with CNN+MLP	UNSW-NB15 [6], CICIDS-2017 [7]	✓	Raw bytes
Di Monda et al. [29] (2024)	Detection of new types of attacks	Few-shot incremental learning using CNN-based feature embeddings with a conventional classifier (SVM, KNN, LR)	CICIDS-2018 [7]	✓	Packet data (biflow)
Zhang et al. [30] (2024)	Sample scarcity and computing resource limitations	Few-shot transfer learning with CNN (feature extraction) + logistic regression classifier	Edge-IIoT [31], USTC2016 [25]	✓	Raw bytes
Seo et al. [32] (2024)	Detection of new types of attacks	Signature-based detection with signature generation for new types	Local collection, IoT-23 [33], CICIDS-2017 [7]	✓	Raw bytes
Our work	Out-of-order packets, class imbalance	Bidirectional analysis and self-supervised learning	WIDE (real traces), QUIC22 [34]	✓	Packet size and time

TABLE II
ATTACK CLASS INFORMATION DEFINED IN THE INTRUSION LOG

Class	Attack Prefixes
HTTP	"alphflHTTP", "ptmpHTTP", "mptpHTTP", "ptmplaHTTP", "mptplaHTTP"
Multi	"ptmp", "mptp", "mptmp"
Alpha	"alphfl", "malphfl", "salphfl", "point_to_point", "heavy_hitter"
IPv6Tunnel	"ipv4gretun", "ipv46tun"
PortScan	"posca", "ptpposca"
ICMPScan	"ntscIC", "dntscIC"
UDPScan	"ntscUDP", "ptpposcaUDP"
TCPScan	"ntscACK", "ntscSYN", "sntscSYN", "ntscTCP", "ntscnull", "ntscXmas", "ntscFIN", "dntscSYN"
DoS	"DoS", "distributed_dos", "ptpDoS", "sptpDoS", "DDoS", "rflat"

TABLE III
CONSTRUCTED DATA WITH CLASS LABELS FROM WIDE TRACE (09/01/2020-09/30/2020)

Class	k=3	k=5	k=10	k=20
HTTP	5,006	3,639	2,502	2,208
Multi	19,478	12,410	6,236	2,866
Alpha	136,070	131,888	27,527	5,525
PortScan	1	1	0	0
DoS	42	33	3	3
Unlabeled	17,963,835	13,252,563	4,075,647	940,004

packet loss and retransmission [10]. Possibly, recovering from packet reordering by referring to the sequence number field is an option, but it is a highly expensive operation with extra space and time requirements in an intermediate point in the network. Such dynamics is generally more critical in wireless mobile communications with greater loss and retransmission rates. Second, the measured data collection shows a high degree of *class imbalance* with only a small fraction of attack connections. We can also find significant variations even from the distribution of attack types (also observed from our dataset in Table III). Another challenge would be a trade-off between the number of packet stream variables to be managed and analyzed vs. the corresponding time/space complexity: for example, a greater set of variables in analysis may yield better performance, while imposing greater overheads and expenses.

This paper sheds light on the design of deep learning models for effective packet stream analysis in the wild, which enables us to make timely responses against network attacks by identifying in their early stages. The key contributions of this study can be summarized, as follows:

- We formulate the problem of packet stream analysis with the notion of time series classification, which optimizes identification performance while minimizing the number of packets on a flow referenced in the analysis process.
- We present two deep sequence models designed to overcome the challenges of incomplete packet sequences and class imbalance: (i) a *bidirectional recurrent structure* designed for greater resilience to missing/out-of-order packet streams, and (ii) a *pre-training-enabled sequence-to-sequence structure* designed for creating consistent representations from unbalanced class distributions using self-supervised learning.
- We share evaluation details with our observations and analysis. We conduct extensive evaluations using three different data collections, one from the Internet backbone

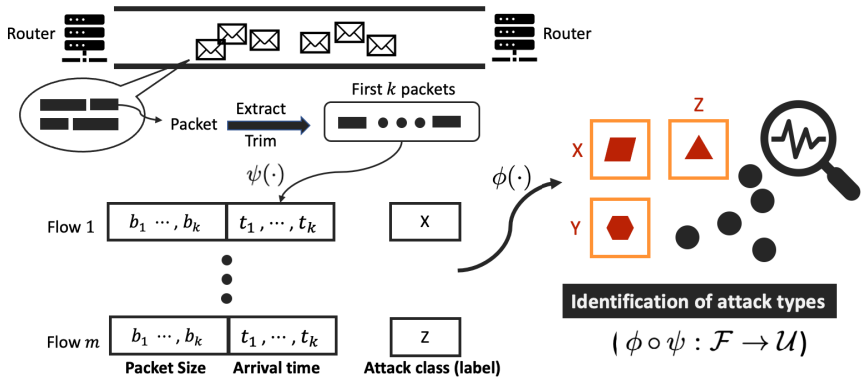


Fig. 1. Early attack identification: Each flow is represented with the size and time information, and the framework performs the identification of attack classes based on the flow representation. Note that $\psi(\cdot)$ is the flow representation function and $\phi(\cdot)$ is the flow identification function that takes the input flow represented by $\psi(\cdot)$ to determine its class.

links and the others collected from simulated networking environments. The experimental results show that our deep sequence models consistently outperform baseline deep learning models across diverse experimental settings, yielding up to 94.8% (F1 score) with the first five packets ($k = 5$) from the Internet traffic collection containing a substantial fraction of network flows experiencing out-of-order delivery.

The organization of this paper is as follows. We begin with providing the description of the problem and a summary of closely related studies in Section II. We next introduce the dataset constructed from the Internet backbone and synthetic traffic traces with the associated ground-truth logs in Section III. Section IV presents our deep sequence models developed to analyze the potentially imperfect and unbalanced packet stream data, with a schematic overview of the the framework model implementing the deep sequence models to tackle the problem of early identification. We share the experimental setting and results with our observations and implications in Section V, conducted with three data collections. We finally conclude our presentation in Section VI, with the summary of the study and future directions.

II. Background

This section provides the description of the problem and our framework model designed to tackle the problem.

A. Problem Description

This study aims to accurately identify the type of the network connections in their early stages by analyzing the stream of packets. To formulate the problem, we employ the general concept of connections and flows defined for point-to-point communications. A connection consists of two flows for bidirectional communication (e.g., client-to-server and server-to-client directions). Let c_i be a connection and $\mathcal{C}$ the set of connections in a captured network trace. Then, the size of flow set $\mathcal{F}$ is twice the corresponding connection set size, i.e., $|\mathcal{F}| = 2 \times |\mathcal{C}|$. A common way of representing a flow (for early identification) is to associate it with a two-dimensional vector containing the *packet size* and *packet time* recorded for each packet within the flow, which has often been considered as time series analysis for traffic classification [11]. For flow f, let $b_j(f)$ be the number of bytes of the j-th packet and $t_j(f)$ be the inter-packet time between the (j-1)-th and j-th packets ($t_1(f) = 0$). Let ψ be the function that transforms a flow into this representation: $\psi(f) = \langle (b_1(f), \ldots, b_n(f)), (t_1(f), \ldots, t_n(f)) \rangle$, for flow f consisting of n packets. Note that $\psi_k(f)$ takes the first k packets in the observation (for the purpose of early decision). Figure 1 depicts the data transformation from a series of packets captured on a communication link.

For attack types, suppose m attack classes under consideration. A flow may either be a part of an attack connection or have nothing to do with any of attack classes (referred to as "background"). Let $\mathcal{U}$ be a set of different classes, $\mathcal{U} = \{A_i | 0 \leq i \leq m, i \in \mathbb{Z}\}$, where A_0 is a class for background traffic while $A_j (j > 0)$ is an attack class. In other words, a flow belongs to $A_i (0 \leq i \leq m)$ depending on its class. The identification function $\phi(\cdot)$ takes the input representation of a flow $\psi(f)$ and determines its class, i.e., $\phi \circ \psi : \mathcal{F} \rightarrow \mathcal{U}$. As shown in Figure 1, the early identification framework performs $\phi(\cdot)$ to characterize attack classes by monitoring the size and time information measured from the first k packets on a flow.

To measure the performance of the identification function, we utilize standard metrics based on the confusion matrix, including F1 score and false positive rate (FPR) (defined in Section V-A). As described, only the first k packets in a flow are monitored for early identification. The goal is then to maximize the identification performance for a given packet constraint k^* as follows:

$$\max \phi(\cdot) \text{ F1 score} \quad \text{subject to} \quad k \leq k^*.$$

For simplicity, we set $k = k^*$, i.e., the number of packets monitored is equal to the constraint.

B. Related Work

1) Intrusion and Anomaly Detection: Intrusion detection is a typical second line of defense mechanism, which can broadly be classified into two groups of signature-based and

anomaly-based approaches [8]. Conventional intrusion detection tools, such as Snort (https://www.snort.org/) and Zeek (https://zeek.org/), have the capability of detecting unwanted connections in real time often using textual byte patterns. Given the wider reliance on payload encryption, however, those signature-based approaches would be less attractive [8]. Moreover, there is an increase of privacy disputes and legal regulations, and the payload inspection may not be a valid option, particularly at a certain network point.

A body of studies investigated various ML approaches for detecting anomalies from network traffic traces [3], [4]. Despite substantial improvement with brand-new methods, anomaly detection is inherently limited to the binary decision without providing information specific to attack classes. Additionally, existing studies largely rely on complete connection information, which only becomes available after the communication is over. Several studies [7], [12], [13] focused on attack identification (rather than anomaly detection). A recent study [14] compares shallow learning and deep learning methods for network attack identification. Using public network datasets, the authors demonstrate that deep learning models, such as convolutional neural network (CNN)-based ones, do not outperform traditional shallow learners like random forest. Another study [15] explores the impact of mislabeled data in network attack identification, proposing a clustering-based classification method with boundary augmentation to better classify difficult samples located near the decision boundary. While these studies provide valuable insights, they do not address the early identification capability that our study aims to achieve.

Another active line of research avenue in this field focuses on online detection at the data plane. Several studies have explored the feasibility of performing detection directly on programmable switches to enable real-time intrusion detection. For example, an early effort in [16] demonstrates the integration of a tree-based classification model into the data plane, addressing the resource constraints of switch hardware. The work in [17] extends online analysis to support line-rate performance (scaling up to multi-terabit speeds), while introducing runtime programmability that allows model updates without requiring a switch reboot. Another work in [18] presents a co-design approach that incorporates deep learning models, particularly recurrent neural networks (RNNs), into the architecture of programmable switches.

2) Packet Stream Analysis: Traffic classification is one of the core functions in network monitoring and management for traffic engineering, resource provisioning, Quality-of-Service (QoS) preservation, and security purposes. Several studies tackled the problem of traffic classification based on packet stream analysis, and hence, their proposed method might provide the function of early classification of network flows by referencing the first few packets. For instance, the study in [35] utilized statistical information, such as mean and variance of packet size and inter-arrival time, for classifying TCP connections by referring to the first k packets. The work in [19] records the packet size difference and inter-arrival time for the first 10 packets on the network flow, which is then converted to an image to apply a CNN-based classification model for traffic classification. Authors in [20] investigated the applicability of multi-task learning, simultaneously performing multiple tasks leveraging information across the tasks, for the purpose of traffic classification using a one-dimensional CNN model. The study by [23] explores packet representation using an attention-based encoding mechanism that can be used for different downstream classification tasks, not based on flows but based on individual packets.

Packet stream analysis for early classification has also been explored in the context of attack detection. In [36], the authors extract features from the initial set of packets to perform binary classification (i.e., benign or malicious). Similarly, the study in [26] uses a one-dimensional CNN to process raw packet contents from the early portion of a flow, followed by a multi-layer perceptron (MLP) for classification, while the authors in [27] focused on updating the classifier model for new types of attacks. Several studies focused on the challenge of limited training data, particularly the scarcity of attack samples. For example, the authors in [28] explore the potential of transfer learning not only to mitigate data scarcity but also to address computational constraints in edge network environments. Similarly, the works in [29] and [37] investigate the use of few-shot learning to handle the scarcity of attack samples, particularly for novel or emerging threats where labeled data is difficult to obtain. A related study in [30] further examines the effectiveness of both few-shot and transfer learning in scenarios constrained by both limited samples and computing resources. In regard to data scarcity, our study considers the challenge of class imbalance, where attack samples represent only a small portion of the training data. While previous research highlights transfer learning as a promising solution, we explore an alternative approach—self-supervised learning. This direction is motivated by recent findings suggesting that pre-training a supervised learning model using unlabeled samples can enhance performance [38]. Moreover, to the best of our knowledge, prior work has not addressed the challenge of out-of-order packet sequences, which are common in real-world network environments and can significantly affect detection accuracy. In this study, we propose a deep sequence modeling approach specifically designed to handle incomplete and potentially disordered packet sequences, thereby enhancing the practicality of early-stage traffic analysis.

Table I summarizes prior research on packet stream analysis, focusing on approaches that utilize packet-level data extracted from flows or biflows, as opposed to aggregated flow-level statistics, to enable early decisions. Several studies [26], [28], [30] employ CNNs to extract features directly from raw traffic bytes. In contrast, our work takes a different direction by avoiding the parsing of raw packet bytes, thereby enabling analysis of encrypted traffic streams. To improve practicality, we leverage only packet size and inter-arrival time information—features that are easy to obtain with light collection and computing complexities. Furthermore, as summarized in the table, previous studies largely rely on conventional back-end classifiers (e.g., MLPs) combined with front-end convolutional networks. In contrast, our approach emphasizes temporal modeling through deep sequence architectures based

on recurrent structures, which are better suited for time-series analysis. Lastly, while existing studies often rely on network traffic data collected from simulated environments, our work leverages real-world traffic captured from Internet backbone links, offering a more realistic and representative foundation for evaluation.

3) Early Detection Studies: Early detection has been a keyword for many application domains, with the goal of minimizing the impact of adversarial events by detecting them in the premature stage. For example, authors in [39] presented a neural network structure combining recurrent and convolutional networks for the early detection of fake news on social media, motivated to reduce societal and economic damages. Similarly, the studies in [40] and [41] are in the loop of rumor detection using graph-structured neural networks. Fraudulent event detection is closely related to the fake news detection, with the focus on the early detection of malicious activities like posting harmful links on social media by using an RNN-based survival analysis [42] and fraudulent reviews through one-class adversarial networks [43]. Recent research efforts extend fake news and rumor detection by introducing sophisticated modeling techniques. For instance, the work in [44] defines a dual convolution networks for capturing the dynamics of messages in propagation and the background knowledge, while the study in [45] designs a multi-view representation method to jointly capture textual and visual features. Additionally, early detection would be highly crucial in the healthcare domain as well, and the authors in [46] presented a survival analysis-based method predicting when a patient develops complications after being diagnosed. Unlike our problem, these studies need *less* attention to the inconsistency in data sequences owing to the assumption of messages on social media posted with time stamps.

III. Packet Stream Data Construction

To identify different classes of attacks, the availability of annotated datasets (with the labeled class information) would basically be required for the unique representation of each class. Many intrusion detection studies relied exhaustively on CSV-formatted connection data provided in, for example, KDD Cup 1999 [47], UNSW-NB15 dataset [6], and CIC-IDS2017 [7], but those connection files contain the information of flows that can be only collected *after* the completion of communication. Hence, such connection logs would rather be useful for postmortem analysis than for early detection of ongoing malicious flows.

In this work, we construct fine-grained packet-level datasets including the size (in bytes) and time information of individual packets (i.e., constructing $\psi(f)$) by combining network traffic traces with corresponding intrusion detection logs. We consider both real and synthetic traces for constructing packet-level datasets. The real traffic (known as the WIDE trace) is captured from Internet backbone links in Japan [48], [49], while the synthetic packet traces are recorded on simulated network environments, including UNSW-NB15 [6] and HIKARI-2021 [50]. Note that UNSW-NB15 provides both packet-level traces and CSV-formatted connection files (compiled from the raw traces), and we consider the packet traces for early identification in this study. In this section, we introduce the process of the packet stream data construction, primarily with the real Internet traces.

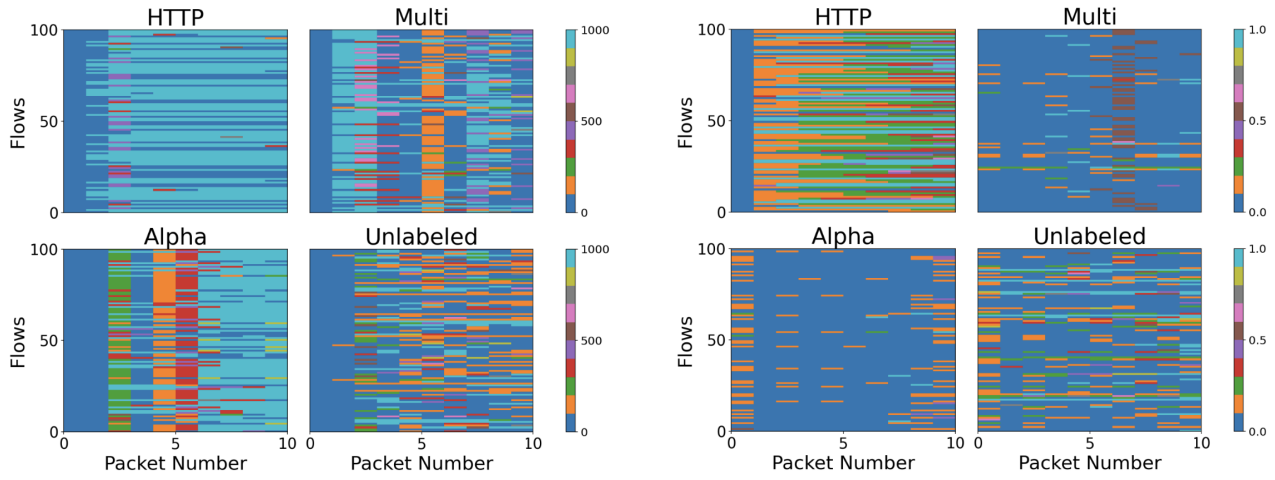
A. Data Construction From Internet Traces

As discussed, we utilize the WIDE trace with the corresponding intrusion logs for labeling of traffic [48], [49]. The traffic trace contains TCP/IP packet header information in a `pcap` file, while the associated intrusion log is provided in a comma-separated values (CSV) file with the attack information inferred by multiple detectors. Each `pcap` file is a recording of 15-minute traffic collected on a specific day. A previous study in [51] constructed network data by combining these traffic traces and intrusion logs, but the resulting dataset contains NetFlow-like statistical information made only available at the connection release time. In this work, we construct packet stream data to develop the function for the early identification of network attacks.

Here, we describe the details of our data construction process. Basically, the intrusion log contains the flow identification information ("5-tuple"), including the IP address pair, source/destination port numbers, and the transport-layer protocol (e.g., TCP). The log also provides the attack class information, including HTTP-relevant attacks ("HTTP"), heavy hitter traffic ("Alpha"), point-to-multipoint anomalies ("Multi"), denial-of-service attacks ("DoS"), port scanning ("PortScan"), network scanning ("ICMPScan", "TCPScan", and "UDPScan"), and tunneling behaviors ("IPv6Tunnel") [49]. As can be seen from Table II, a specific attack is categorized to one of the attack classes based on its intrinsic characteristics.

Using the flow identifier, the traffic trace is combined with the associated intrusion log. If the flow from the traffic trace has a match to a log entry, then the flow is marked as an attack with the class information ($A_i \in \mathcal{U} \wedge i \neq 0$). If no match is found, then the combining procedure falls back to 4-tuple (excluding the source port number information from 5-tuple) and performs the same combining task. In case of no match again from the fall-back process, the flow is considered background traffic ($A_0 \in \mathcal{U}$). The statistical packet stream information is then recorded with the class information for each flow in the traffic trace. In this work, we consider TCP flows only since it is safe to construct a flow using connection management flags (e.g., a SYN packet for a new flow start).

Table III shows the extracted flow information from the 25-day network traffic collected in September 2020, excluding five days due to unavailability (3rd, 14th, 27th, 28th, and 29th). In the table, we use the label of "Unlabeled" to indicate the background traffic (i.e., flows that have not been assigned to any of the attack classes during the combining phase). From the one-month trace, we observe a set of flows in five attack classes in addition to background traffic, while there exists no flow for the rest of the attack classes. Note that it is natural that the number of flows decreases with greater k values, because there can be a subset of flows consisting of a small number of packets only, and any flow with less than k packets should be excluded from the collection.



(a) Visualization of packet lengths over 10 packets (in bytes)

(b) Visualization of inter-packet times over 10 packets (in seconds)

Fig. 2. Visualization of the packet size and time for 100 random samples (WIDE): Each row shows a flow and each column shows a time step (packet number) from 1 to 10. Each class shows somewhat distinctive patterns across time steps.

TABLE IV
BACKGROUND TRAFFIC STATISTICS (TOP-5)

	Source port		Destination port	
Rank	Port number	Share	Port number	Share
1	443 (HTTPS)	19.4%	443 (HTTPS)	12.3%
2	22 (SSH)	8.4%	445 (SMB)	11.1%
3	80 (HTTP)	5.4%	22 (SSH)	7.5%
4	25 (SMTP)	1.7%	80 (HTTP)	4.7%
5	3389 (RDP)	1.3%	23 (Telnet)	3.3%

TABLE V
CONSTRUCTED DATA WITH CLASS LABELS (UNSW-NB15)

Class	k=3	k=5	k=10	k=20
Fuzzers	817,255	357,470	38,247	8,482
Exploits	2,052,650	192,933	25,357	2,308
DoS	993,114	38,597	3,672	310
Generic	379,282	26,003	3,055	267
Reconnaissance	361,017	35,093	4,144	315
Analysis	200,705	28,980	883	4
Worms	4,189	2,091	250	17
Backdoors	42,611	1,198	95	10
Shellcode	24,439	610	0	0
Unlabeled	1,407,101	1,303,604	952,061	697,613

Table IV provides additional details about the background traffic, including the distribution of source and destination ports. While HTTPS accounts for the largest share of TCP traffic, standard HTTP remains widely used in web communications. Secure Shell (SSH) contributes to the second largest portion of the traffic. We observe traffic asymmetry in both directions, which may be attributed to factors such as traffic engineering, load balancing, asymmetric routing, and multi-path forwarding. This observation supports our assumption regarding the importance of out-of-order packet sequences for effective traffic characterization.

As described in Section II-A, we primarily collect the packet length $b_j(f)$ and inter-packet time $t_j(f)$ information for the characterization. Figure 2 shows the visualization of the sampled flows in different classes, with respect to $b_j(f_i)$ (Figure 2(a)) and $t_j(f_i)$ (Figure 2(b)), where $i = [1, 100]$ and $j = [1, 10]$. Although randomly sampled, we can see somewhat distinctive patterns for individual classes across time steps, which signals the feasibility of the identification process in their early stage.

Finally, the extracted data samples are pre-processed before being fed to ML algorithms, which largely includes rescaling and data normalization. Collectively, they constitute $\phi(\cdot)$. In particular, we observed high skewness from both the byte and inter-packet time fields. To resolve this, in the rescaling process, logarithm functions were applied to take the skewness of distributions into account: $\log(b_j(f))$ and $\log(t_j(f))$. Then, data normalization was performed with the min-max scaling function: $\{\log(\cdot) - \min(\log(\cdot))\}/\{\max(\log(\cdot)) - \min(\log(\cdot))\}$.

B. Data Construction From Simulated Traffic

For comprehensive experiments, we further construct two additional datasets derived from the synthetic traffic recorded from simulated network environments.

1) UNSW-NB15 [6]: This dataset was created in 2015 from a simulated environment consisting of servers for traffic generations and routers capturing packets (and saving them to `pcap` files). The simulated network contains two servers for the normal spread of the traffic and one server for generating malicious traffic. This dataset defines nine attack classes: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. The ground-truth log information was obtained using the Bro-IDS tool (https://www.zeek.org/) The dataset contains two collections, each of which contains a single data trace captured in January and February, respectively. We construct the packet stream data from the January collection.

Table V shows the number of instances for each class, which shows the majority of classes (other than Unlabeled, Fuzzers, and Exploits) contain a small number of instances only. In the

TABLE VI
CONSTRUCTED DATA WITH CLASS LABELS (HIKARI-2021)

Class	k=3	k=5	k=10	k=20
Bruteforce	5,145	5,884	5,884	4,924
Bruteforce-XML	5,884	5,145	5,144	859
Probing	23,388	23,388	23,385	23,289
Unlabeled	2,271,659	2,258,299	570,998	144,280

table, Unlabeled indicates background traffic that has not been assigned to any of the attack classes, as in Table III.

2) HIKARI-2021 [50]: This dataset was published in 2021. The network configuration shows an attacker network, a victim network, and an interface to the public Internet. Like the WIDE and UNSB-NB15 data, HIKARI-2021 provides the raw packet traces in the `pcap` format with the ground-truth data. The collection defines four attack types: Bruteforce, Bruteforce-XML, Probing, and XMRIGCC (malicious crypto-mining). The non-attack data contains both simulated benign traffic and background traffic to/from the public network. From our data construction using the entire traces and ground-truth, we found no instance of XMRIGCC. Table VI shows the number of instances for each class constructed from the HIKARI-2021 collection.

IV. METHODS—DEEP SEQUENCE MODELS

The primary goal of this study is to develop an effective packet stream analysis tool that provides accurate early attack identification from potentially incomplete, skewed packet stream datasets with out-of-order sequences and unbalanced class distributions. In this section, we initially describe the overview of deep learning for time series analysis, and then present our deep sequence models designed to address the above challenges: (i) bidirectional long short-term memory (LSTM) as an implementation of bidirectional recurrent structure, and (ii) sequence-to-sequence autoencoder as an implementation of pre-training-enabled sequence-to-sequence structure.

A. Deep Learning for Time Series Analysis

Considering a sequence of packets in a flow as the input variable and a class as the target variable, we can formulate the identification problem as a classification problem in a supervised learning setting. In particular, we aim to learn a function ϕ_Θ, which predicts an attack class or the background class given the first k packets of a flow: $A_i = \phi_\Theta(\psi_k(f))$, where Θ is a set of model parameters to be learned; and $\psi_k(\cdot)$ maps the flow representation to k tuples.

Among various forms of ϕ_Θ,[1] we choose recurrent neural networks (RNNs) because they are specifically designed for processing time series data by utilizing shared and recurring units [52]. At time index t, RNNs operate on an input x_t and update hidden states $\boldsymbol{h}_t = (h_t^{(1)}, h_t^{(2)}, \ldots)$ by applying a recurring function,

$$\boldsymbol{h}_t = \mathrm{RNN}(x_t, \boldsymbol{h}_{t-1}; \Theta_{\mathrm{RNN}}),$$

[1]We assume that ϕ_Θ also includes pre-processing of data samples: rescaling and data normalization.

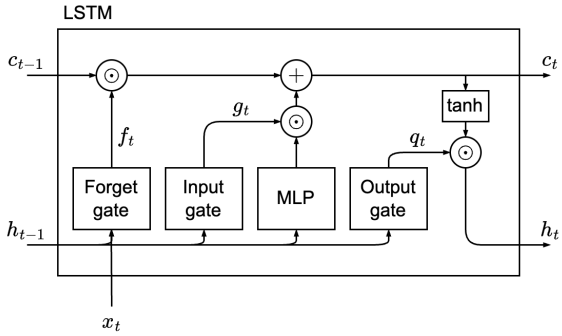


Fig. 3. **LSTM cell information flow**. At the tth time step, the LSTM cell takes the previous hidden state $\boldsymbol{h}_{t-1}$ and the previous cell state $\boldsymbol{c}_{t-1}$. The forget gate, $\boldsymbol{f}_t$, controls the information flow from the previous cell state, the input gate, $\boldsymbol{g}_t$, controls the information flow to update the cell state, and the output gate $\boldsymbol{q}_t$ controls the information flow from the current cell state $\boldsymbol{c}_t$ to the output $\boldsymbol{h}_t$. The operator $\odot$ denotes the element-wise multiplication.

where Θ_{RNN} denotes the model parameters of RNN and $h_t^{(\ell)}$ denotes a hidden state at the ℓth RNN layer. For the classification task, a classifier network takes (a part of) the last hidden state $\boldsymbol{h}_{\mathrm{last}}$ to make a prediction on a probability distribution over a set of classes, i.e., $\boldsymbol{p} = \mathrm{c}_\xi(\boldsymbol{h}_{\mathrm{last}})$, where $\boldsymbol{p} = [p_0, \ldots, p_m]^\mathsf{T}$ denotes a vector of probability masses; each p_i denotes the probability that the flow is classified as the ith label; and c_ξ is a classifier network with a set of trainable parameters ξ. In our scenario, we pre-process $\psi_k(f)$ and set it to x_t. For a given $\boldsymbol{p}$, the attack (or background) class A_i is determined by the index i that satisfies $p_i \geq p_j$ for all $j \in \{0, \ldots, m\}$.

1) Long Short-Term Memory (LSTM): As standard RNNs typically suffer from vanishing or exploding gradient problems [53], modern RNN architectures such as LSTM [54] and gated recurrent unit (GRU) [55] were proposed as gating mechanisms to mitigate such problems by creating computational paths whose gradients do not vanish or explode for a long period. In this study, we choose LSTM over GRU as LSTM is known to be strictly more expressive, outperforming GRU in many complex applications [56]. LSTM resolves vanishing/exploding gradient problems by adding LSTM cells. In addition to the standard RNN hidden states $\boldsymbol{h}_t$, LSTM cells have a cell state $\boldsymbol{c}_t$, which is updated by an internal recurrence and creates paths where the gradient can flow over longer duration. Then the gating mechanisms control the flow of information by employing three gates: input, output, and forget gates. The input gate controls the extent to which new input data is transferred to update $\boldsymbol{c}_t$, the output gate controls the extent to which the LSTM cell output flows out to the output (the hidden state), and the forget gate controls the extent to which $\boldsymbol{c}_t$ remains unchanged. The gating mechanisms allow important information to last longer while forgetting less significant information. Now, the recurring function becomes

$$\boldsymbol{h}_t, \boldsymbol{c}_t = \mathrm{LSTM}(x_t, \boldsymbol{h}_{t-1}, \boldsymbol{c}_{t-1}; \Theta_{\mathrm{LSTM}}).$$

Figure 3 illustrates the information flow of the LSTM cell.

2) Bidirectional LSTM: The use of bidirectional LSTM represents a novel approach to network traffic modeling, especially in scenarios with missing or out-of-order elements

in the input sequence. While RNNs, including LSTM and GRU, have been explored for traffic classification tasks [57], they often struggle when sequence data is incomplete [58] or unordered [59]. Traditional LSTM models rely on single-direction processing and typically require customized pre-processing, such as interpolating missing values based on domain-specific heuristics [58], which may not be adaptable across different applications.

In contrast, bidirectional LSTM provides a more advanced solution by processing information bidirectionally (both forward and backward) within a sequence. This dual-directional analysis enhances the model's ability to infer context around missing or misaligned data points, leading to higher accuracy without requiring application-specific data handling techniques [60]. By offering improved contextual awareness and more robust dependency modeling [61], [62], bidirectional LSTM can capture crucial relationships across sequential data, even when elements are out of order. This approach positions the bidirectional LSTM as a promising advancement for resilient and adaptable traffic classification in diverse network conditions.

In our task of the attack classification, the performance of the task depends not only on understanding "causal" relationship of packets in the input sequence, but also on extracting other useful features that would otherwise be overlooked by only focusing on causality. However, a regular LSTM only employs a past history to extract features. To address this issue, bidirectional LSTM [63] has been studied and has demonstrated its effectiveness in many applications [64]. Thus, in this study, we consider the bidirectional LSTM, which combines two LSTM architectures: one moving forward from the beginning to the end of the input sequence and the other moving backward from the end to the beginning of the input sequence. That is,

$$\boldsymbol{h}_t^{\text{fwd}}, \boldsymbol{c}_t^{\text{fwd}} = \text{LSTM}(x_t, \boldsymbol{h}_{t-1}^{\text{fwd}}, \boldsymbol{c}_{t-1}^{\text{fwd}}; \Theta_{\text{LSTM}}^{\text{fwd}}),$$
$$\boldsymbol{h}_t^{\text{bwd}}, \boldsymbol{c}_t^{\text{bwd}} = \text{LSTM}(x_t, \boldsymbol{h}_{t+1}^{\text{bwd}}, \boldsymbol{c}_{t+1}^{\text{bwd}}; \Theta_{\text{LSTM}}^{\text{bwd}}),$$

where the superscripts $^{\text{fwd}}$ and $^{\text{bwd}}$ indicate the quantities associated with the forward and backward operations. The final hidden representation at time t can be obtained by the concatenation, $\boldsymbol{h}_t = [\boldsymbol{h}_t^{\text{fwd}}, \boldsymbol{h}_t^{\text{bwd}}]$. With this architecture, we anticipate that the model can handle a flow better even if packets arrive out of order.

3) Training: As the classifier network is designed to produce a probability that the input flow is categorized into each class, the last component of the classifier is a softmax layer, which turns its input vector $\boldsymbol{o}$ into the probability vector $\boldsymbol{p}$ such that $p_i = \exp(o_i)/\sum_i \exp(o_i)$. Then, the model can be trained by minimizing the cross-entropy loss:

$$L_{\text{CE}} = -\frac{1}{n_{\text{train}}} \sum_{j=1}^{n_{\text{train}}} \sum_{i=0}^{m} y_{j,i} \log(p_{j,i}),$$

where $y_{j,i} \in \{0, 1\}$ is an indicator denoting if the index i of a training sample j is the correct classification; $p_{j,i}$ is the p_i of the training sample j; and n_{train} is the number of training samples.

B. Self-Supervised Learning

In many applications, the distribution of classes of interest could be biased or skewed, and training a classifier directly could fail to capture an underlying distribution correctly. To mitigate this class imbalance issue, a training algorithm that does not require the knowledge of class labels can be employed to pre-train the classifier. As pointed out in the state-of-the-art unsupervised learning study (contrastive-loss-based models, e.g., [38]), a main purpose of unsupervised learning is to pre-train a model, which is in turn fine-tuned with supervised learning. We follow this approach in this study.

1) Sequence-to-Sequence Autoencoder: Autoencoder is a feed-forward neural network, which attempts to copy its input sequence to its output sequence. We consider a type of autoencoder, called sequence-to-sequence (Seq2Seq) autoencoder model, whose input and output are sequences. In general, the Seq2Seq model consists of three parts: an encoder, a decoder, and a context vector. To handle the sequence, the encoder and the decoder are typically modeled as RNN-type neural networks. The encoder processes an input sequence and produces the context vector, which would contain the most salient features of the input sequence. Figure 4 depicts the network architecture of the Seq2Seq autoencoder model. Internally, the multilayer perceptron (MLP) component inside the encoder compresses the output of the LSTM to produce the context vector projected in a lower-dimensional space, and the other MLP at the decoder performs the decompression.

We now formally describe the Seq2Seq model. Specifically, the model can be written in a form:

$$\tilde{x} = g_{\text{dec}}(g_{\text{enc}}(x; \Theta_{\text{enc}}); \Theta_{\text{dec}}),$$

where g_{enc} and g_{dec} denote the encoder and the decoder, respectively; Θ_{enc} and Θ_{dec} denote the model parameters for the encoder and the decoder, respectively; $\tilde{x}$ denotes the reconstruction of x that is a pre-processed version of $\psi_k(f)$. The encoder takes the input sequence and produces the context vector c:

$$c = g_{\text{enc}}(x; \Theta_{\text{enc}}).$$

The decoder performs the reverse action of the encoder,

$$\tilde{x} = g_{\text{dec}}(c; \Theta_{\text{dec}}),$$

where the output approximates the input $\tilde{x} \approx x$.

2) Training: The goal of the training is to find sets of model parameters $(\Theta_{\text{enc}}, \Theta_{\text{dec}})$ that minimize mean squared error (MSE) between the output sequence and the input sequence:

$$L_{\text{MSE}} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \|x_i - \tilde{x}_i\|_2^2,$$

where x_i and $\tilde{x}_i$ denote the i-th training sample for the input and the reconstruction, respectively.

Note that this approach can be categorized as self-supervised learning, as it does not require additional labels for training, except the input variable. Once the training of the Seq2Seq autoencoder is finished, we take the network weights and biases of the encoder as initial values of network weights and biases of the LSTM network for the classification,

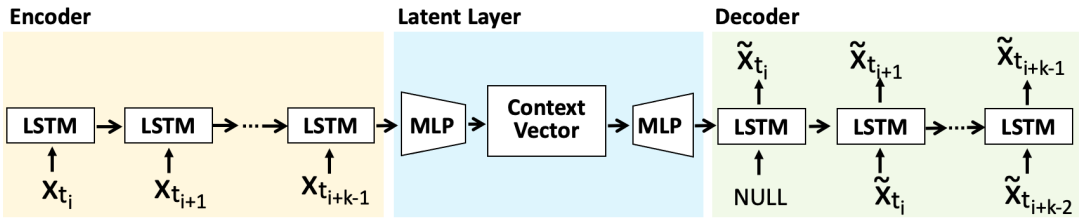


Fig. 4. Seq2Seq autoencoder for self-supervised learning: The encoder and the decoder are typically modeled as RNN-type neural networks to handle the sequence. Internally, the MLP component inside the encoder compresses the output of the LSTM to produce the context vector projected in a lower-dimensional space, and the other MLP at the decoder performs the decompression to restore the sequence.

i.e., $\Theta_{\text{RNN}} = \Theta_{\text{enc}}$. After that, we continue training the LSTM network to minimize the cross-entropy explained in Section IV-A.

The mathematical ground as to why self-supervised learning can be more robust in learning representations in the scenarios of class imbalance is still an active area of research. However, strong empirical evidence has been presented in a recent work where self-supervised learning resulted in more robust representations [65].

C. Proposed Framework

To realize early identification based on the deep sequence method, we design a framework consisting of a set of functional components to fulfill its mission, as illustrated in Figure 5. The following are the details about the framework components:

- *Data Gen* creates normalized, labeled data instances by combining the given packet traces and the associated intrusion logs, which will be used for constructing the learning model. Attack flows are labeled with their specific attack class, while the rest of the flows are labeled as "Unlabeled";
- *Packet Stream Gen* groups a stream of packets based on the flow identifier information. The statistical packet stream information from the predetermined number of packets (k) on the same flow is measured and delivered to *Early Identifier*;
- *Pre-trainer* is equipped to perform self-supervised learning, which helps initialize the neural network in a way to stabilize the learning model from the impact by probabilistic skewness and class imbalance. Note that the label information is not referenced in the pre-training stage;
- *Main Trainer* learns the characteristics of individual attack classes by referencing the annotated class label information. The pre-trained model is loaded as the initialization of the Main Trainer, and the labeled samples are injected to construct the learning model for early attack identification;
- *Early Identifier* performs the identification of attack classes against the given data instance delivered from *Packet Stream Generator*. The learning model built by *Main Trainer* is loaded to this component for actual decisions. The early decision is then made by classifying each instance into one of the classes ($A_i \in \mathcal{U}$).

Basically, there are two parts in the presented framework. The data measurement part (*Data Gen* and *Packet Stream Gen*) described in Section III. In fact, the ML part is the core of the framework performing learning and decisions. As mentioned, there would be several challenges for realizing early decisions from potentially imperfect and unbalanced network packet data. We develop two deep sequence models for addressing those challenges, as discussed earlier in this section. The designed sequence-based models include a bidirectional recurrent structure built upon long short-term memory (LSTM) for resiliency against packet missing and reordering (designed for *Main Trainer*), and a pre-training-enabled sequence-to-sequence structure based on the autoencoding architecture for stability from unbalanced class distributions (designed for *Pre-Trainer*). The framework has been implemented using PyTorch and we will discuss the evaluation of the proposed method in the following section.

In Figure 5, the process flow is as follows: (i) The framework starts by creating labeled training data from raw packet traces and the associated intrusion logs; (ii) The labeled data is then fed into the deep sequence model, which characterizes each attack class using the first k packets from each sample; (iii) Self-supervised learning may be employed to enhance model initialization, making the neural network distribution-aware; (iv) The deep sequence model (bidirectional LSTM or Seq2Seq) is trained to learn the unique features of each class; (v) The trained model is subsequently deployed for real-time testing on individual flows captured from live traffic; (vi) Flow information is extracted based on five-tuple identification from the raw traffic; (vii) Finally, the deep sequence model makes a classification decision based on the extracted flow data.

V. Evaluation

In this section, we share the evaluation results with our observations and implications. We first describe the experimental setting for compared models, data preparation, and evaluation metrics. Then we report the experimental results, which demonstrate the feasibility of our deep sequence models for early decisions with improved performance compared to the baseline models. This section primarily reports the experimental results collected with the WIDE dataset. Additionally, we share the results obtained with the synthetic datasets (UNSW-NB15 and HIKARI-2021) and the (UDP-based) QUIC dataset (CESNET-QUIC22 [34]).

A. Experimental Setting

We evaluate a set of deep learning models for packet stream analysis in the context of network attack identification. As the

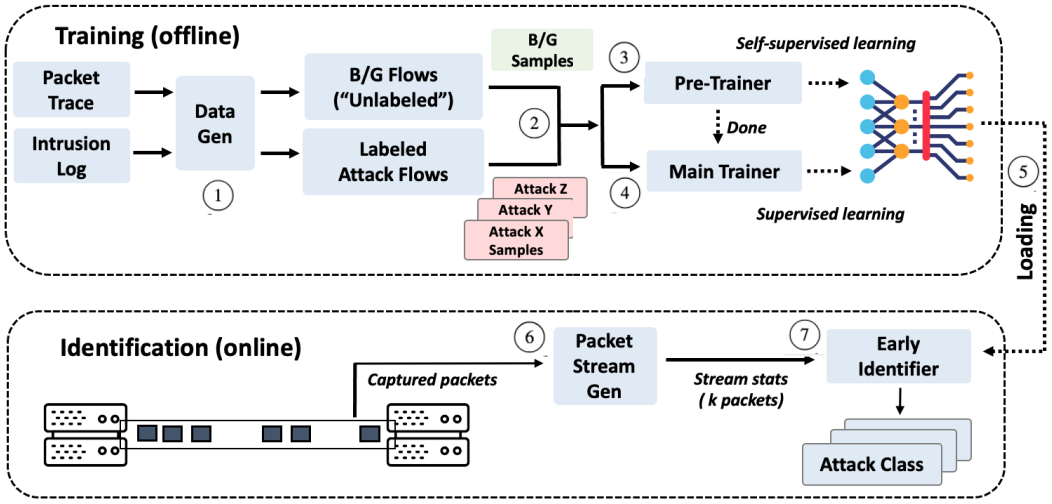


Fig. 5. Overview of the proposed framework for early attack identification, which analyzes packet streams using a time series-aware ML component for learning and classifying. The circled numbers represent the process flow in the framework.

baseline, we consider two neural network models: (i) a multi-layer perceptron (MLP) model that takes a vector containing the packet size and time without the notion of time sequences ("MLP") and (ii) a unidirectional LSTM model widely applied for time series analysis ("FWD"). As the realization of the proposed models, we implement two deep sequence models: (iii) the bidirectional LSTM model ("BI") and (iv) the unidirectional LSTM model pre-trained with Seq2Seq autoencoder ("SEQ"). Note that we configure FWD with the identical neural network architecture to SEQ, with the same level of optimizations for fair comparison; hence, SEQ can be regarded as a pre-trained version of FWD with self-supervised learning. For BI, we test models with an additional fully-connected layer that produces a hidden vector, which has the same size as the context vector. Internally, we use `ReLU` for MLP and `tanh` for LSTM for non-linear activation. For each ML model, we utilize check-pointing to search a model instance with the greatest validation accuracy over 1,000 epochs with the learning rate 10^{-3}.

As shown in Table III, the created dataset mainly contains three attack classes (HTTP, Multi, and Alpha). We composed the ML models to classify the traffic into four classes (i.e., three attack classes and the background). For training, $\min(10000, x)$ number of instances were randomly chosen from each class pool, where x is the actual size of the pool. Hence, the sampled dataset is intrinsically unbalanced with less than or equal to 10,000 samples, particularly if k gets larger. The validation set contains 200 instances for each class, and the testing set includes 100 instances per class. To ensure no-overlap among training, validation, and test sets, the validation and test samples had been drawn before composing the training set.

To measure model performance, we basically refer to the confusion matrix consisting of TP (True Positive), FP (False Positive), FN (False Negative), and TN (True Negative). We measure the identification performance using *F1 score*, since the metric of *accuracy* may lead to a biased outcome under a setting with unbalanced classes (same as our setting). The metric of F1 score is defined as: F1 score $= \frac{2TP}{2TP+FP+FN}$, and hence, the model performs better if the score is higher. Since the attack identification is *not* a binary classification problem, we calculate F1 score based on the number of true instances for each class (i.e., *Macro Avg.* F1 score). We additionally report false positive rate (FPR) defined as: FPR $= \frac{FP}{FP+TN}$. In our setting, a false positive event refers to a case where Unlabeled (A_0) is classified into one of the attack classes ($\{A_i | i \neq 0\}$).

Our implementation and the WIDE dataset can be accessible online through: https://github.com/dcstamuc/PacketStreamDataCollection

B. Performance Comparison

We first perform a fair comparison under the assumption of the identical configuration setting for the number of hidden layers (HLAYER) and the hidden dimension size (HDIM): HLAYER $= 3$ and HDIM $= 20$. Additionally, we set the size of context vector (or hidden vector) (CV) to three for BI and SEQ as their default values (i.e., CV $= 3$). We then report the configuration of each model yielding the best F1 score with the corresponding FPR. Finally, we analyze the performance of BI and SEQ with different values for CV and HDIM.

Figure 6 shows the classification performance in F1 score under the fair comparison setting. From the figure, we can see that our deep sequence models (BI and SEQ) substantially outperform MLP consistently across different k's. Interestingly, the unidirectional LSTM model (FWD) collapses to 0.1 when $k = 10$, implying high sensitivity and instability of the model when the sampled dataset gets more unbalanced. BI and SEQ are capable of providing early identification of attack flows with fairly acceptable performance even with three packets ($k = 3$), while monitoring the first five packets ($k = 5$) escalates the identification performance to 93.5% (BI) and 93.7% (SEQ). One interesting observation is that increasing the number of packets ($k \geq 10$) does not help improve the identification performance, which will be discussed in Section V-F.

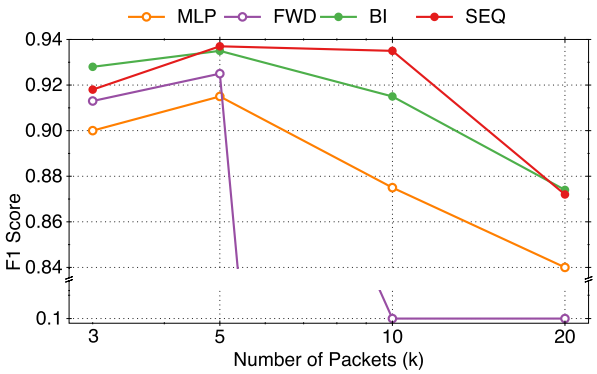


Fig. 6. Performance comparison: BI and SEQ consistently outperform MLP and FWD across different k values, yielding up to 93.5% (BI) and 93.7% (SEQ) with $k = 5$. Note that FWD collapses to around 10% when $k \geq 10$ (thus showing the broken line).

TABLE VII
BEST-PERFORMING CONFIGURATION FOR EACH MODEL ($k = 5$)

Model	HLAYER	CV	HDIM	F1 Score	FPR
MLP	3	–	40	0.937	0.13
FWD	3	–	40	0.945	0.07
BI	3	10	40	0.947	**0.05**
SEQ	3	10	20	**0.948**	0.08

TABLE VIII
PERFORMANCE (F1 SCORE) UNDER DIFFERENT CONFIGURATIONS (BOLD=BEST, ITALIC= BEST FOR THE OTHER MODEL)

Model	k=3	k=5	k=10	k=20
BI(3,20)	0.928	0.935	0.915	0.874
BI(5,20)	0.935	0.935	0.908	0.878
BI(10,20)	0.938	0.945	0.935	0.888
BI(3,40)	**0.948**	0.943	0.930	**0.910**
BI(5,40)	0.923	0.935	**0.938**	0.896
BI(10,40)	0.933	*0.947*	0.923	0.883
SEQ(3,20)	0.918	0.937	0.935	0.872
SEQ(5,20)	0.915	0.947	0.918	0.869
SEQ(10,20)	0.915	**0.948**	0.928	*0.901*
SEQ(3,40)	*0.925*	0.943	**0.938**	0.893
SEQ(5,40)	0.915	0.925	0.906	0.893
SEQ(10,40)	0.915	0.922	0.916	0.866

We next compare the best performance of the models. In fact, we have performed extensive hyper-parameter-search on a grid and here we report the results of the best configurations with respect to F1 score when $k = 5$ (as observed in Figure 6). Table VII provides the performance with F1 score and FPR for each model. In the table, CV is not applicable for MLP or FWD. The results show that SEQ is slightly better than the others, while BI shows the lowest FPR. Although the performance gap between the baseline models (MLP and FWD) and our deep sequence models (BI and SEQ) is small here, BI and SEQ perform more consistently, which is a crucial factor in actual deployment.

Lastly, we provide further results for BI and SEQ under different CV and HDIM settings. For the exposition purpose, we name a model with a suffix of (CV, HDIM): for example, BI(3,20) refers to the BI model with CV = 3 and HDIM = 20.

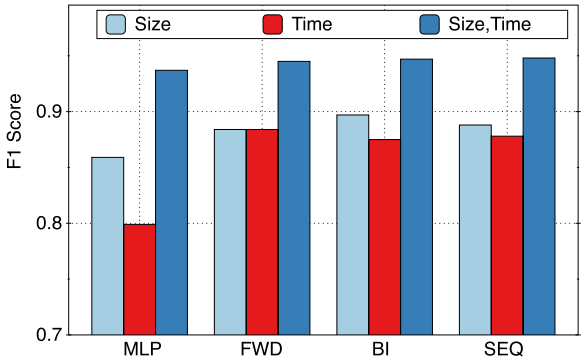


Fig. 7. Performance comparison for different feature sets ($k = 5$): Utilizing both features perform highly consistently across different identification models. Relying on the size information shows slightly better performance than utilizing the time information that produces an F1 score of at least 79.9%.

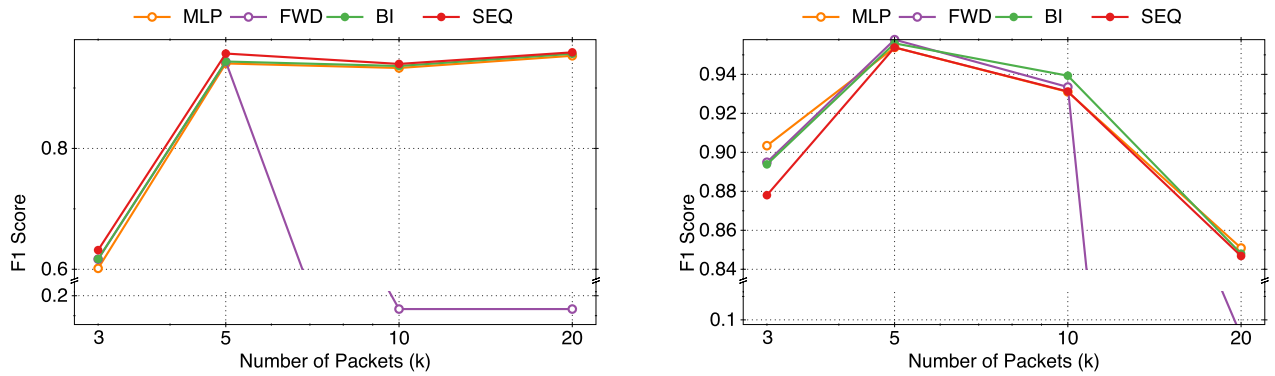
The results in Table VIII suggest a room to optimize the models with non-negligible improvements compared to the default setting, i.e., (CV = 3, HDIM = 20). In the table, the bold-faced result indicates the best performance among the entire settings for a given k. We also italicize the best performance of the other model for the comparison of the two models (i.e., BI and SEQ). For example, BI(3,40) yields the best performance (and bold-faced) when $k = 3$, while SEQ(3,40) works the best among the SEQ settings (and italicized). Overall, BI(3,40) performs consistently, yielding over 91% without any significant variations across different configurations. SEQ works well by and large but shows some degraded performance with the longest sequence ($k = 20$), presumably due to the increased complexity of the reconstruction.

C. Significance of Packet Size and Time Features

Thus far, we assumed a two-dimensional vector containing the packet size and packet time as input for characterizing a flow f, i.e., $\langle (b_1(f), t_1(f)), \ldots, (b_k(f), t_k(f)) \rangle$, from k packets measured for that flow. As shown in Figure 2, individual classes may have distinctive patterns when visualizing the sample instances using the size and time information, respectively. To see their impact on identification performance, we conduct another experiments with a single variable sequences (i.e., either size or time).

Figure 7 shows the performance comparison with different feature sets: packet size information only (i.e., $\langle b_1(f), b_2(f), \ldots, b_k(f) \rangle$), packet time information only (i.e., $\langle t_1(f), t_2(f), \ldots, t_k(f) \rangle$), and both the size and time information. The number of packets is set to five ($k = 5$) in this comparison study. The figure shows that using both features leads to highly consistent performance across different identification models. Relying solely on the size information shows slightly better performance than utilizing only the time information that produces an F1 score of 79.9% in the worst case. This result supports the significance of utilizing both the size and time information for packet stream analysis.

Table IX and Table X provide further details of the measured performance when using the packet size information only (Table IX) and the packet time information only (Table X). We



(a) UNSW-NB15 (b) HIKARI-2021

Fig. 8. Performance comparison with other datasets: BI and SEQ do not show substantial performance improvements with the synthetic traffic sets that have relatively low rates of out-of-order delivery compared to the WIDE dataset, as shown in Table XI.

TABLE IX
PERFORMANCE WITH PACKET SIZE INFORMATION

Model	k=3	k=5	k=10	k=20
MLP	0.856	0.859	0.884	0.847
FWD	0.859	0.884	0.1	0.1
BI	0.871	0.897	0.905	0.834
SEQ	0.849	0.888	0.933	0.845

TABLE X
PERFORMANCE WITH PACKET TIME INFORMATION

Model	k=3	k=5	k=10	k=20
MLP	0.770	0.799	0.760	0.788
FWD	0.798	0.884	0.1	0.1
BI	0.811	0.875	0.851	0.830
SEQ	0.800	0.878	0.848	0.814

TABLE XI
FRACTION OF FLOWS EXPERIENCING OUT-OF-ORDER PACKETS

WIDE	UNSW-NB15	HIKARI-2021
17.96%	2.2%	5.19%

can see from both tables that our deep sequence models overall outperform the baseline models, yielding over 80% of performance across different k values consistently. In particular, the SEQ model trained with the packet size information shows an F1 score of 93.3% when $k = 10$, approximating to the best result in Table VIII. We also observed that FWD collapses with poor performance with $k \geq 10$, as in the previous experiments.

D. Experiments With Synthetic Traces

Here, we carry out experiments with the synthetic collections of UNSW-NB15 and HIKARI-2021. The experimental setting is identical to what was used for the WIDE collection analysis (described in Section V-A).

For UNSW-NB15, we selected three classes of Fuzzers, Exploits, and Unlabeled for evaluating our identification function since other classes contain only a small number of flows when k is large. In Table V, DoS contains only 310 samples when $k = 20$, while other attacks than Fuzzers and Exploits have an even smaller number of instances than DoS has. Figure 8(a) shows the identification performance over different k values. We can see that $k = 3$ would not be enough to accurately classify, while $k \geq 5$ show significantly improved performance. Note that FWD failed when $k = 10, 20$, similar to what we observed in Figure 6.

We next report experimental result conducted with the HIKARI-2021 collection. We experimented with the four classes defined in Table VI. The performance comparison in Figure 8(b) shows somewhat closer patterns to the WIDE evaluation result in Figure 6. All the models perform the best with $k = 5$, while referring to more packets reduces the chance of correct identification.

The experimental results with the synthetic collections (Figure 8(a) and Figure 8(b)) show that BI and SEQ do not make any substantial performance enhancement, unlike the observation from WIDE (Figure 6). We conjecture that this is because the synthetic traffic was collected in highly controlled networking settings.

We measured packet losses from the three data collections by analyzing packet sequence numbers. Table XI compares the three datasets with respect to the fraction of flows experiencing out-of-order packets. The real Internet trace (WIDE) shows considerably greater losses. The HIKARI-2021 collection partly includes the Internet traffic as well as the simulated traffic, which results in a higher loss rate than the entirely synthetic UNSW-NB15 trace. *This result supports our hypothesis that the presented deep sequence models (BI and SEQ) are beneficial for characterizing packet streams in the wild with out-of-order packets in the measurement.*

E. Experiments With QUIC Traffic Dataset

QUIC is a transport protocol built on top of UDP, designed to improve performance for HTTPS traffic [66]. With features like encryption and low latency, this protocol has quickly gained popularity, supporting a variety of applications such as web browsers (Chrome) and video streaming services

TABLE XII
QUIC TRAFFIC CLASSIFICATION PERFORMANCE (F1 SCORE)

	$k = 20$		$k = 30$	
Model	Validation	Testing	Validation	Testing
MLP	0.786	0.750	0.754	0.732
FWD	0.1	0.1	0.1	0.1
BI	0.842	0.806	0.823	0.795
SEQ	0.827	0.829	0.809	0.798

(YouTube) [67], [68]. The rapid adoption of QUIC led to increased share in the Internet traffic and over 8% of websites worldwide are QUIC-capable as of November 2024 [69]. However, this growing adoption also raises security concerns, as the relatively new protocol may still have vulnerabilities, including susceptibility to DoS attacks on QUIC-enabled servers [70]. In this study, we evaluate our deep sequence models with QUIC traffic to assess the applicability of our proposed method.

The CESNET-QUIC22 dataset [34] provides a month-long QUIC traffic captured from Internet backbone links. The dataset includes bidirectional connection data, featuring packet sequence information (such as packet sizes, directions, and inter-packet times) for the first 30 packets, along with aggregated connection statistics (including duration, transmitted bytes and packets, and packet histograms). The dataset defines 18 class labels, and we chose four majority classes (Streaming, Social, Advertising, and Search) having the highest number of data instances for our experiments.

Table XII compares the performance of models for QUIC traffic classification when $k = 20$ and $k = 30$. For evaluation, we employed the model configuration that had performed the best in Section V-B without extra optimizations. Additionally, we excluded the use of meta information (connection statistics), relying solely on the packet size and inter-packet time features. The results indicate that the proposed models (BI and SEQ) outperform the baseline models (MLP and FWD) in both settings. The FWD model shows poor performance, collapsing to 0.1, which is consistent with previous experiments. Our proposed models show at least a 5% improvement, with no significant difference observed between BI and SEQ.

F. Discussion

Now, we discuss some interesting observations from our experiments with the WIDE dataset. In Figure 6, we observed that increasing the number of packets ($k \geq 10$) does not improve any identification performance, which is somewhat counter-intuitive since monitoring more packets could give further information about the communication. Regarding this, we can get some clues from previous studies. A pioneering study in [35] claims that the first four packets of a TCP connection would be sufficient to characterize network applications, and the authors observed that monitoring more than four packets even results in the degradation of identification performance. A recent study in [20] also concludes that keeping track of more packets does not help the prediction accuracy for traffic classification, in which k ranged between 30 and 60.

Another interesting observation in our evaluation is the impact of training samples on identification performance. As described, we randomly sampled $\min(10000, x)$ number of instances from each class pool for the evaluation (including 200 for validation and 100 for testing), where x is the actual size of the pool. In Table XIII, we organize four different sets for evaluation: Set1 has 1,000 instances sampled and Set2 10,000 instances sampled from the first week, while Set3 has 1,000 samples and Set4 10,000 samples from the whole month. With the week-long datasets (Set1 and Set2), we can clearly see the benefit of the increased training set size. In contrast, the month-long setting does not give any progress even with the larger dataset. One possible explanation is that using 10,000 instances would not be enough to capture the distribution of data from the month-long pool, while that same number would suffice for the relatively small week-long pool. This would also be a reason why Set2 shows the better performance than Set4. However, if we have to draw a less number of instances (i.e., 1,000 samples for Set1 and Set3), the result shows that sampling from the whole month performs better; presumably, it would be beneficial to choose samples from the larger (month-long) pool if we have to choose only a small number of instances. We observed almost the same patterns from different BI and SEQ configurations. In fact, this study limits the number of samples to 10,000 to keep computational complexity manageable but evaluating the models with an extensive set of data samples from a longer-term capture would be interesting to better understand the impact of data quantity and quality.

TABLE XIII
F1 SCORE WITH DIFFERENT DATASET CONFIGURATIONS (k=5):1,000 SAMPLES FOR SET1 & SET3 AND 10,000 FOR SET2 & SET4

	Week (09/01 – 09/07)		**Month (09/01 – 09/30)**	
Model	Set1	Set2	Set3	Set4
BI(10,20)	0.913	0.968	0.947	0.945
SEQ(10,20)	0.895	0.955	0.948	0.948

VI. CONCLUSION

This paper presented the design and evaluation of deep sequence models designed for effective packet stream analysis to achieve accurate identification of network attacks in a timely manner. With the rationale of network dynamics (causing out-of-order packet sequences) and class imbalance (resulting in learning inconsistent representations), we designed two deep sequence models based on a bidirectional structure (for the former concern) and a pre-training-enabled Seq2Seq structure (for the latter concern). To develop and evaluate the deep sequence models, we construct packet-level datasets from the network traffic traces collected from the Internet backbone links and simulated networking environments. Our experimental results support the feasibility of the presented deep sequence models, showing up to 94.8% (F1 score) for identifying three different attack types (with one background traffic class) from the Internet traffic collection containing a substantial fraction of network flows that experienced out-of-order delivery. The results from the three data collections also suggest that monitoring the first five packets ($k = 5$) would suffice, enabling timely responses against network threats while the connection is still active and in progress.

There are several directions for further investigations. We examined the two deep sequence models independently, but ultimately the models would be combined to expect a synergistic benefit, which will be the next step of this study. Data quantity would be a concern for analyzing the data collected over an extended period (e.g., several years). Since the learning complexity is proportionate to data quantity, discovering a more represented group of data samples would be interesting and statistical approaches could be investigated for that purpose. In this study, we considered the number of packets as the criterion for early attack identification. Another criterion would be a time constraint (e.g., making a decision within t seconds), and it will be interesting to explore neural sequence structures to support such an alternative criterion. Additionally, we plan to apply the deep sequence approach for conventional traffic classification and application identification to see its feasibility in other network monitoring tasks.

ACKNOWLEDGMENT

The authors wish to express their sincere gratitude to the anonymous reviewers for their insightful and constructive feedback. This work was supported in part by the East Texas A&M University Presidential GAR Initiative program.

REFERENCES

[1] Q. Zou, A. Singhal, X. Sun, and P. Liu, "Deep learning for detecting network attacks: An end-to-end approach," in *Proc. IFIP Conf. Data Appl. Secur. Priv. (DBSec)*, 2021, pp. 221–234.

[2] A. Oprea, Z. Li, R. Norris, and K. Bowers, "MADE: Security analytics for enterprise threat detection," in *Proc. 34th Annu. Comput. Secur. Appl. Conf.*, Dec. 2018, pp. 124–136.

[3] N. Moustafa, J. Hu, and J. Slay, "A holistic review of network anomaly detection systems: A comprehensive survey," *J. Netw. Comput. Appl.*, vol. 128, pp. 33–55, Feb. 2019.

[4] G. Fernandes, J. J. P. C. Rodrigues, L. F. Carvalho, J. F. Al-Muhtadi, and M. L. Proença, "A comprehensive survey on network anomaly detection," *Telecommun. Syst.*, vol. 70, no. 3, pp. 447–489, Mar. 2019.

[5] L. Dhanabal and S. Shantharajah, "A study on NSL-KDD dataset for intrusion detection system based on classification algorithms," *Int. J. Adv. Res. Comput. Commun. Eng.*, vol. 4, no. 6, pp. 446–452, 2015.

[6] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Nov. 2015, pp. 1–6.

[7] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.

[8] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, Dec. 2019.

[9] H. X. Nguyen and M. Roughan, "Rigorous statistical analysis of internet loss measurements," *IEEE/ACM Trans. Netw.*, vol. 21, no. 3, pp. 734–745, Jun. 2013.

[10] A. El-Atawy, Q. Duan, and E. Al-Shaer, "A novel class of robust covert channels using out-of-order packets," *IEEE Trans. Dependable Secure Comput.*, vol. 14, no. 2, pp. 116–129, Mar. 2017.

[11] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1988–2014, 2nd Quart., 2019.

[12] C. Zhang, X. Costa-Perez, and P. Patras, "Tiki-taka: Attacking and defending deep learning-based intrusion detection systems," in *Proc. ACM SIGSAC Conf. Cloud Comput. Secur. Workshop*, Nov. 2020, pp. 27–39.

[13] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[14] A. Lichy, O. Bader, R. Dubin, A. Dvir, and C. Hajaj, "When a RF beats a CNN and GRU, together—A comparison of deep learning and classical machine learning approaches for encrypted malware traffic classification," *Comput. Secur.*, vol. 124, Jan. 2023, Art. no. 103000.

[15] Q. Yuan et al., "BoAu: Malicious traffic detection with noise labels based on boundary augmentation," *Comput. Secur.*, vol. 131, Aug. 2023, Art. no. 103300.

[16] B. M. Xavier, R. S. Guimarães, G. Comarela, and M. Martinello, "Programmable switches for in-networking classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2021, pp. 1–10.

[17] S. U. Jafri, S. Rao, V. Shrivastav, and M. Tawarmalani, "Leo: Online ML-based traffic classification at multi-terabit line rate," in *Proc. USENIX Symp. Netw. Syst. Des. Impl. (NSDI)*, 2024, pp. 1573–1591.

[18] Z. Zhao, Z. Li, Z. Song, F. Zhang, and B. Chen, "RIDS: Towards advanced IDS via RNN model and programmable switches co-designed approaches," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2024, pp. 591–600.

[19] Z. Chen, K. He, J. Li, and Y. Geng, "Seq2Img: A sequence-to-image based approach towards IP traffic classification using convolutional neural networks," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2017, pp. 1271–1276.

[20] S. Rezaei and X. Liu, "Multitask learning for network traffic classification," in *Proc. 29th Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2020, pp. 1–9.

[21] S. Rezaei and X. Liu, "How to achieve high classification accuracy with just a few labels: A semisupervised approach using sampled packets," in *Proc. Ind. Conf. Data Min. (ICDM)*, 2019, pp. 28–42.

[22] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.

[23] X. Meng, Y. Wang, R. Ma, H. Luo, X. Li, and Y. Zhang, "Packet representation learning for traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Min.*, 2022, pp. 3546–3554.

[24] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. H. Lashkari, "Detection of DoH tunnels using time-series classification of encrypted traffic," in *Proc. IEEE Int. Conf. Depend. Auton. Secure Comput. Int. Conf. Pervasive Intell. Comput. Int. Conf. Cloud Big Data Comput. Int. Conf. Cyber Sci. Technol. Congr. (DASC/PiCom/CBDCom/CyberSciTech)*, 2020, pp. 63–70.

[25] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.

[26] T. Ahmad, D. Truscan, J. Vain, and I. Porres, "Early detection of network attacks using deep learning," in *Proc. IEEE Int. Conf. Softw. Test., Verification Validation Workshops (ICSTW)*, Apr. 2022, pp. 30–39.

[27] J. Zhang, F. Li, and F. Ye, "Sustaining the high performance of AI-based network traffic classification models," *IEEE/ACM Trans. Netw.*, vol. 31, no. 2, pp. 816–827, Apr. 2023.

[28] D. A. Bierbrauer, M. J. De Lucia, K. Reddy, P. Maxwell, and N. D. Bastian, "Transfer learning for raw network traffic detection," *Expert Syst. Appl.*, vol. 211, Jan. 2023, Art. no. 118641.

[29] D. Di Monda, A. Montieri, V. Persico, P. Voria, M. De Ieso, and A. Pescapè, "Few-shot class-incremental learning for network intrusion detection systems," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 6736–6757, 2024.

[30] X. Zhang et al., "Enhanced few-shot malware traffic classification via integrating knowledge transfer with neural architecture search," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5245–5256, 2024.

[31] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

[32] H. Seo and M. Yoon, "Generative intrusion detection and prevention on data stream," in *Proc. USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 4319–4335.

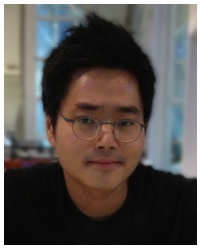
[33] A. Parmisano, S. Garcia, and M. J. Erquiaga, "A labeled dataset with malicious and benign IoT network traffic (1.0.0)," Zenodo, 2020. [Online]. Available: https://doi.org/10.5281/zenodo.4743746

[34] J. Luxemburk, K. Hynek, T. Čejka, A. Lukačovič, and P. Šiška, "CESNET-QUIC22: A large one-month QUIC network traffic dataset from backbone lines," *Data Brief*, vol. 46, Feb. 2023, Art. no. 108888.

[35] L. Bernaille, R. Teixeira, and K. Salamatian, "Early application identification," in *Proc. Int. Conf. Emerg. Netw. Exp. Technol. (CoNEXT)*, 2006, pp. 1–12.

[36] I. Guarino, G. Bovenzi, D. Di Monda, G. Aceto, D. Ciuonzo, and A. Pescapé, "On the use of machine learning approaches for the early classification in network intrusion detection," in *Proc. IEEE Int. Symp. Meas. Netw. (M&N)*, Jul. 2022, pp. 1–6.
[37] G. Bovenzi, D. Di Monda, A. Montieri, V. Persico, and A. Pescapè, "Classifying attack traffic in IoT environments via few-shot learning," *J. Inf. Secur. Appl.*, vol. 83, Jun. 2024, Art. no. 103762.
[38] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9729–9738.
[39] Y. Liu and Y.-F. Wu, "Early detection of fake news on social media through propagation path classification with recurrent and convolutional networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 354–361.
[40] X. Yang, Y. Lyu, T. Tian, Y. Liu, Y. Liu, and X. Zhang, "Rumor detection on social media with graph structured adversarial learning," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, Jul. 2020, pp. 1417–1423.
[41] T. Bian et al., "Rumor detection on social media with bi-directional graph convolutional networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020, pp. 549–556.
[42] P. Zheng, S. Yuan, and X. Wu, "SAFE: A neural survival analysis model for fraud early detection," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 1, pp. 1278–1285.
[43] P. Zheng, S. Yuan, X. Wu, J. Li, and A. Lu, "One-class adversarial nets for fraud detection," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 33, 2019, pp. 1286–1293. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/3924
[44] M. Sun, X. Zhang, J. Zheng, and G. Ma, "DDGCN: Dual dynamic graph convolutional networks for rumor detection on social media," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 36, 2022, pp. 4611–4619.
[45] Q. Ying, X. Hu, Y. Zhou, Z. Qian, D. Zeng, and S. Ge, "Bootstrapping multi-view representations for fake news detection," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2022, pp. 5384–5392.
[46] B. Liu, Y. Li, Z. Sun, S. Ghosh, and K. Ng, "Early prediction of diabetes complications from electronic health records: A multi-task survival analysis approach," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 32, 2018, pp. 101–108.
[47] M. Ahmed, A. Naser Mahmood, and J. Hu, "A survey of network anomaly detection techniques," *J. Netw. Comput. Appl.*, vol. 60, pp. 19–31, Jan. 2016.
[48] R. Fontugne, P. Borgnat, P. Abry, and K. Fukuda, "MAWILab: Combining diverse anomaly detectors for automated anomaly labeling and performance benchmarking," in *Proc. ACM Conf. Emerg. Netw. Exp. Technol.*, Nov. 2010, pp. 1–12.
[49] J. Mazel, R. Fontugne, and K. Fukuda, "A taxonomy of anomalies in backbone network traffic," in *Proc. Int. Wireless Commun. Mobile Comput. Conf. (IWCMC)*, Aug. 2014, pp. 30–36.
[50] A. Ferriyan, A. H. Thamrin, K. Takeda, and J. Murai, "Generating network intrusion detection dataset based on real and encrypted synthetic attack traffic," *Appl. Sci.*, vol. 11, no. 17, p. 7868, Aug. 2021.
[51] J. Kim, C. Sim, and J. Choi, "Generating labeled flow data from MAWILab traces for network intrusion detection," in *Proc. ACM Workshop Syst. Netw. Telemetry Anal.*, Jun. 2019, pp. 45–48.
[52] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by error propagation," in *Parallel Distributed Processing, Volume 1: Explorations in the Microstructure of Cognition: Foundations*. The MIT Press, Jul. 1986. [Online]. Available: https://doi.org/10.7551/mitpress/5236.003.0012
[53] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, Mar. 1994.
[54] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
[55] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1724–1734.
[56] D. Britz, A. Goldie, M.-T. Luong, and Q. Le, "Massive exploration of neural machine translation architectures," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2017, pp. 1442–1451.
[57] M. Camelo, P. Soto, and S. Latré, "A general approach for traffic classification in wireless networks using deep learning," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 5044–5063, Dec. 2022.
[58] Z. C. Lipton, D. C. Kale, and R. C. Wetzel, "Directly modeling missing data in sequences with RNNs: Improved classification of clinical time series," in *Proc. Mach. Learn. Healthc. Conf.*, 2016, pp. 253–270.
[59] O. Vinyals, S. Bengio, and M. Kudlur, "Order matters: Sequence to sequence for sets," in *Proc. 4th Int. Conf. Learn. Represent.*, May 2016, pp. 1–11.
[60] S. Siami-Namini, N. Tavakoli, and A. S. Namin, "The performance of LSTM and BiLSTM in forecasting time series," in *Proc. IEEE Int. Conf. Big Data*, Dec. 2019, pp. 3285–3292.
[61] R. L. Abduljabbar, H. Dia, and P.-W. Tsai, "Development and evaluation of bidirectional LSTM freeway traffic forecasting models using simulation data," *Sci. Rep.*, vol. 11, no. 1, p. 23899, Dec. 2021.
[62] X. Lu, L. Yuan, R. Li, Z. Xing, N. Yao, and Y. Yu, "An improved bi-LSTM-based missing value imputation approach for pregnancy examination data," *Algorithms*, vol. 16, no. 1, p. 12, Dec. 2022.
[63] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.
[64] A. Graves, A.-r. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, May 2013, pp. 6645–6649.
[65] H. Liu, J. Z. HaoChen, A. Gaidon, and T. Ma, "Self-supervised learning is more robust to dataset imbalance," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021, pp. 1–24.
[66] A. Langley et al., "The QUIC transport protocol: Design and internet-scale deployment," in *Proc. Conf. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2017, pp. 183–196.
[67] T. Shreedhar, R. Panda, S. Podanev, and V. Bajpai, "Evaluating QUIC performance over web, cloud storage, and video workloads," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 2, pp. 1366–1381, Jun. 2022, doi: 10.1109/TNSM.2021.3134562.
[68] A. Bujari, C. E. Palazzi, G. Quadrio, and D. Ronzani, "Emerging interactive applications over QUIC," in *Proc. IEEE 17th Annu. Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2020, pp. 1–4.
[69] *Usage Statistics of QUIC for Websites*. Accessed: Nov. 5, 2024. [Online]. Available: https://w3techs.com/technologies/details/ce-quic
[70] E. Chatzoglou, V. Kouliaridis, G. Karopoulos, and G. Kambourakis, "Revisiting QUIC attacks: A comprehensive review on QUIC security and a hands-on study," *Int. J. Inf. Secur.*, vol. 22, no. 2, pp. 347–365, Apr. 2023.

Minji Kim received the master's degree in computer science from the Computer Science Department, East Texas A&M University. Her research interests include network traffic analytics, deep sequence models, healthcare informatics, time series analysis, and machine learning.



Dongeun Lee received the B.S. degree in computer science and engineering and the Ph.D. degree in electrical engineering and computer science from Seoul National University, Seoul, South Korea, in 2006 and 2014, respectively. He was a Computer Systems Engineer with the Lawrence Berkeley National Laboratory and also jointly affiliated as a Post-Doctoral Research Associate with the School of Electrical and Computer Engineering, Ulsan National Institute of Science and Technology, Ulsan, South Korea. He joined the Department of Computer Science, East Texas A&M University, in 2016, where he is currently an Associate Professor. His current research interests include big data analytics for time-series data and scientific machine learning.

Kookjin Lee received the B.S. and M.S. degrees from Seoul National University and the Ph.D. degree in computer science from the University of Maryland, College Park. He is currently an Assistant Professor with the Computer Science Department, School of Computing and Augmented Intelligence, Arizona State University. Before joining ASU, he was a Post-Doctoral Researcher with the Sandia National Laboratories. He received the NSF CAREER Award in 2024.

Doowon Kim (Member, IEEE) received the Ph.D. degree from the University of Maryland in 2020. He is currently an Assistant Professor in electrical engineering and computer science with the University of Tennessee, Knoxville. He conducts research on computer and web security, with particular expertise in phishing attacks and network threats. His contributions to cybersecurity have been recognized through the NSF CAREER Award in 2025, the ACM CCS Distinguished Paper Award in 2025, and the NSA Best Scientific Cybersecurity Paper Award in 2017.

Jinpyo Kim received the master's degree in computer science from the Computer Science Department, East Texas A&M University. His research interests include network traffic analytics, temporal point processes, neural instantiation of stochastic methods, and machine learning.

Sangman Lee received the master's degree in mathematical data science from Korea University, Sejong-si. He is currently an Industry-Academia Collaboration Professor in AI cyber security at Korea University. He works as the CEO of Seqrypton and the Director of the Korea Institute of Information Security & Cryptography. His research interests include intrusion detection systems, network security, cryptography, and quantum circuit optimization.

Jinoh Kim (Senior Member, IEEE) received the Ph.D. degree in computer science from the University of Minnesota, Twin Cities. He is currently a Professor in computer science with East Texas A&M University, an Affiliate Faculty Scientist with the Lawrence Berkeley National Laboratory (LBNL), and an Executive Member of the Silicon Valley Cybersecurity Institute (SVCSI). His research spans distributed systems, network security, IoT/CPS, toward intelligent, resilient, and autonomic systems using AI/ML methodologies. He is a member of ACM.